

Practitioner's Guide for Hybrid AI Development in Data-Sensitive Domains

Paul Sigloch¹ Christoph Benz Müller^{1,2} David Fuenmayor¹

¹ University of Bamberg, Bamberg, Germany

² Free University of Berlin, Berlin, Germany

paul.sigloch@uni-bamberg.de,
christoph.benzmueller@uni-bamberg.de,
david.fuenmayor@uni-bamberg.de

Contents

1	Introduction	2
2	AI Architecture Patterns and Selection	2
2.1	Design Criteria and Constraints Across Architectural Levels	2
2.2	AI Paradigm Selection	4
2.3	Hybrid AI Strategy Selection	6
3	Hybrid AI Strategies	7
4	Development Lifecycle Details	10

1 Introduction

This practitioner’s guide accompanies the research paper “Neuro-Symbolic Verification of LLM Outputs for Data-Sensitive Domains” and provides implementation-oriented guidance for developing hybrid AI systems in data-sensitive domains. While the main paper and supplementary material present the theoretical framework, core principles, and empirical evaluation, this guide offers:

- Concrete decision criteria for AI paradigm selection
- Step-by-step development lifecycle guidance
- Actionable checklists for each development phase

The guide is intended for software engineers, AI practitioners, and system architects who seek practical guidance beyond these theoretical foundations. It is organized as follows. Section 2 presents an incremental methodology for AI architecture design, including criteria for paradigm selection and hybrid integration strategies. Section 3 provides detailed implementation techniques and example applications for each hybrid AI strategy. Section 4 presents the development lifecycle phases with practical checklists of key activities for each framework component.

2 AI Architecture Patterns and Selection

Designing hybrid AI systems requires an incremental methodology that aligns architectures with specific application requirements. The approach progresses through four distinct phases: (1) analyzing design criteria and constraints at component, module, and system levels; (2) selecting appropriate AI paradigms for individual components; (3) evaluating the potential benefits of hybrid approaches at module or system levels; and (4) determining the optimal architectural pattern for integrating symbolic and sub-symbolic components.

The system design criteria and architectural patterns presented here build upon established AI paradigms and integration mechanisms. These recommendations derive from research in hybrid AI architectures [Sarker, 2021; Sarker, 2022; Garcez *et al.*, 2008; Kambhampati, 2020] and practical insights from the HAIMEDA application development.

2.1 Design Criteria and Constraints Across Architectural Levels

Before selecting AI paradigms, practitioners must systematically evaluate the specific requirements that will shape their architectural decisions. The following three categories of criteria should be assessed for each component, module, and the overall system.

2.1.1 Data Characteristics

Understanding the nature of input and output data is crucial for paradigm selection. Symbolic AI excels with structured, rule-based data, while sub-symbolic approaches handle ambiguous, high-dimensional data more effectively. These characteristics directly determine which AI paradigm can process the information most efficiently:

- Input data type (e.g., mathematical symbols, structured forms, natural language)
- Output data type requirements (e.g., binary decisions, structured records, free-form text)
- Data variety (homogeneous vs. heterogeneous)
- Data length variation (consistent vs. highly variable)
- Data ambiguity level (well-defined vs. requiring interpretation)

2.1.2 Performance Requirements

System performance constraints often dictate paradigm choice. Symbolic systems typically offer faster inference and deterministic behavior, while sub-symbolic systems may require more computational resources but provide greater flexibility. These requirements must be balanced against available resources:

- Time constraints (milliseconds vs. seconds vs. minutes)
- Precision requirements (exact vs. approximate acceptable)
- Consistency demands (deterministic vs. probabilistic acceptable)
- Resource limitations (memory, CPU, or GPU constraints)

2.1.3 Domain Constraints

Domain-specific constraints often dictate which architectural options are viable. Symbolic approaches excel when explainability and privacy compliance are mandatory because they offer transparent, deterministic data handling. Operational requirements, such as on-device processing of sensitive data, further narrow the available choices. These non-functional requirements often take precedence over technical considerations when selecting a paradigm:

- Privacy considerations (Personally Identifiable Information (PII) handling, confidentiality requirements)
- Regulatory compliance needs (auditability, explainability)
- Data locality restrictions (on-premises vs. cloud processing)
- Fault tolerance requirements (error handling, recovery mechanisms)

2.2 AI Paradigm Selection

2.2.1 Component-Level Criteria

Careful consideration is required when selecting an appropriate AI paradigm for each individual component or submodule within a larger system. The criteria presented in Table 1 are primarily intended for component-level decisions, where each criterion addresses the specific requirements of discrete functional units. While these criteria are formulated for individual component selection, they may subsequently inform broader architectural considerations when aggregated across multiple components or when evaluating system-wide paradigm choices. The table delineates eleven selection criteria for each AI paradigm to facilitate decision-making at various levels of system design.

Table 1: Selection criteria for choosing AI paradigms at the component level.

Symbolic AI	Sub-Symbolic AI
► Well-defined, constrained problem space with clear rules	► Processing natural language with complex linguistic nuances
► Complete explainability and traceability of decisions essential	► Pattern recognition in high-dimensional data (images, audio, sensors)
► Deterministic outputs with guaranteed consistency required	► Generating creative or novel content based on patterns
► Formal verification needed (safety-critical domains)	► Handling ambiguous requests requiring contextual understanding
► Operations involve mathematical calculations or formal logic	► Generalizing across similar but previously unseen examples
► Structured data with explicit rule application	► Dealing with highly variable or unstructured inputs
► Fast processing with minimal latency critical	► Adapting to evolving patterns in data over time
► Working with exact matches or strict patterns	► Processing inputs where rules are difficult to encode
► System must function with minimal training data	► Extracting implicit relationships from data
► Reference data stable and encodable as rules	► Abstracting high-level concepts from low-level features
► Privacy compliance through deterministic data handling	► Acceptable privacy trade-offs for learning benefits

2.2.2 Module and System-Level Hybrid Criteria

Following the determination of appropriate AI paradigms for individual components, consideration must be given to the architectural decisions at the module and system levels. The question arises whether maintaining consistency with a single AI paradigm throughout a module or system would be preferable, or whether the adoption of a hybrid AI solution would yield superior outcomes. Such considerations are of particular importance, as the implementation of hybrid AI systems may not be warranted in all contexts. These systems introduce greater architectural complexity and require additional resource allocation. Consequently, the decision to employ hybrid approaches requires thorough evaluation to ensure that the anticipated benefits sufficiently justify the associated trade-offs.

The following criteria may be employed to inform decisions regarding the adoption of hybrid AI paradigms at the module or system level:

- The applicability of more than three criteria from both AI paradigms, as delineated in Table 1
- The requirement for cohesive processing of both structured and unstructured data within the system
- The necessity for capabilities encompassing both creative generation and rigorous verification processes
- The need to establish an equilibrium between operational flexibility and formal guarantees
- The presence of processing pipelines that encompass both well-defined procedures and stages characterized by ambiguity
- Domain requirements that necessitate both natural language understanding and formal reasoning capabilities
- The existence of complex workflows exhibiting varying degrees of certainty across different processing stages
- The criticality of maintaining information integrity and factual accuracy whilst preserving naturalness in system outputs

2.3 Hybrid AI Strategy Selection

After determining that a hybrid AI approach offers advantages for one or more modules, practitioners must select appropriate integration strategies. This phase addresses how symbolic and sub-symbolic components should be combined to achieve optimal system behavior. The choice of integration strategy significantly impacts system architecture, performance characteristics, and maintenance requirements.

Integration strategies can be applied at various architectural levels: within a single module, across multiple cooperating modules, or throughout the entire system. Furthermore, these strategies are not mutually exclusive. Sophisticated systems may benefit from combining multiple integration approaches, such as using pre-processing in one module while employing parallel processing in another, or layering strategies to create multi-stage verification pipelines.

Table 2 presents three fundamental integration strategies, each suited to different operational requirements and system objectives. These strategies provide concrete guidance for architectural implementation. Additional implementation techniques and domain-specific examples are detailed in Section 3.

Table 2: Integration strategies for hybrid AI architectures and their selection criteria.

Strategy	Primary Use Cases
1: First Symbolic, Then Sub-Symbolic (Pre-Processing Integration)	► Enhancing generation quality through structured inputs ► Optimizing performance by filtering irrelevant data ► Ensuring security by sanitizing sensitive information ► Standardizing inputs for consistent processing
2: First Sub-Symbolic, Then Symbolic (Post-Processing Integration)	► Content generation with verification requirements ► Extracting structured data from unstructured sources ► Creative solutions requiring validation against constraints ► Natural language outputs that must adhere to specific formats
3: Symbolic and Sub-Symbolic in Parallel (Integrated Processing)	► Complex decision processes requiring multiple reasoning modes ► Interactive systems with dynamic adjustment needs ► Workflows combining creativity and formal constraints ► Problems requiring continuous verification during solution development

3 Hybrid AI Strategies

Once a hybrid approach has been selected, practitioners must determine how to integrate symbolic and sub-symbolic components. The following three strategies represent fundamental integration patterns, each suited to different operational requirements. These strategies are not mutually exclusive, sophisticated systems may combine multiple approaches across different modules or processing stages.

Strategy 1: First Symbolic, Then Sub-Symbolic (Pre-Processing Integration)

- **Primary Use Cases:**
 - Enhancing generation quality through structured inputs
 - Optimizing performance by filtering irrelevant data
 - Ensuring security by sanitizing sensitive information
 - Standardizing inputs for consistent processing
- **Implementation Techniques:**
 - Rule-based input preparation and structuring
 - Constraint-based query formulation
 - Symbolic reasoning to identify relevant context
 - Pre-filtering of data based on formal criteria
 - Input validation against domain schemas
 - Knowledge graph traversal for context enrichment
 - Template-based prompt construction with dynamic variables
- **Example Applications:**
 - Clinical decision support with validated patient data inputs
 - Data analysis pipelines with structurally verified inputs
 - Privacy-preserving NLP with PII detection and anonymization
 - Enhanced search systems with query optimization

Strategy 2: First Sub-Symbolic, Then Symbolic (Post-Processing Integration)

- **Primary Use Cases:**
 - Content generation with verification requirements
 - Extracting structured data from unstructured sources
 - Creative solutions requiring validation against constraints
 - Natural language outputs that must adhere to specific formats
- **Implementation Techniques:**
 - Sub-symbolic generation followed by symbolic verification
 - Pattern extraction via LLMs with rule-based validation
 - Fact-checking through knowledge base comparison
 - Format enforcement through grammar-based parsing
 - Entity recognition followed by logical consistency checking
 - Numerical value extraction with mathematical validation
- **Example Applications:**
 - Medical report generation with protocol compliance verification
 - Legal document drafting with regulatory constraint checking
 - Financial analysis with mathematical consistency validation
 - Technical documentation with terminology standardization

Strategy 3: Symbolic and Sub-Symbolic in Parallel (Integrated Processing)

- **Primary Use Cases:**
 - Complex decision processes requiring multiple reasoning modes
 - Interactive systems with dynamic adjustment needs
 - Workflows combining creativity and formal constraints
 - Problems requiring continuous verification during solution development
- **Implementation Techniques:**
 - Constrained decoding with symbolic guidance during generation
 - Tool-augmented generation with function calling
 - Neuro-symbolic reasoning networks
 - Iteration loops with symbolic evaluation and regeneration
 - Hybrid planning systems with symbolic goals and neural execution
 - Dynamic prompt construction based on symbolic state
 - Multi-agent systems with specialized symbolic and neural agents
- **Example Applications:**
 - Autonomous planning systems with safety constraints
 - Scientific discovery combining hypothesis generation and verification
 - Interactive programming assistants with code correctness checking
 - Complex diagnostic systems combining heuristics and pattern recognition

4 Development Lifecycle Details

Hybrid AI system development follows a five-phase lifecycle that supports iterative refinement through repeated cycles. This framework emerged from the development of the HAIMEDA application and embodies core principles of modularization, iterative development, and privacy-by-design. The recommended approach proceeds from Phase 1 through Phase 4, then repeats Phases 2–4 as needed until the system meets deployment criteria in Phase 5. After deployment, practitioners should periodically revisit Phases 2–5 to address evolving requirements.

Each phase is structured into framework elements designated by alphanumeric identifiers following the convention **E X.Y**, where *X* denotes the phase number and *Y* the element number within that phase. Elements consist of principles (brief foundational statements) and guidelines (concrete implementation advice), with key activities provided alongside each element.

Phase 1: Assessment & Planning

This initial phase requires high rigor as it forms the foundation for all subsequent phases. It establishes key variables that shape the organizational environment and determines which tools, libraries, APIs, and frameworks will guide development.

E 1.1 Domain & Requirements Analysis

E 1.1 Principles:

- **Organize Requirements by Priority**

Create a structured requirements list that identifies core functionalities and establishes clear data handling boundaries based on domain sensitivity. This ensures the system meets functional needs while adapting to evolving requirements and data protection needs.

- **Expect Requirements to Change**

Establish protocols for capturing evolving stakeholder needs throughout the project. Requirements in hybrid AI systems frequently shift as stakeholders better understand AI capabilities and limitations, and as new possibilities emerge during implementation.

E 1.1 Guidelines:

During stakeholder interviews, focus on understanding data privacy constraints, integration requirements, and AI capability expectations. Provide clear explanations of AI techniques and their limitations to ensure realistic understanding. Document both explicit requirements and implicit assumptions, particularly regarding AI performance.

For data-sensitive domains, create a formal classification matrix identifying information that requires special handling. Map workflows where AI could augment human decisions, noting where verification or explanation is needed. This mapping reveals

where symbolic approaches provide necessary transparency or where sub-symbolic methods handle complex patterns.

Requirements will evolve throughout development, whether from changing stakeholder needs or when AI components underperform expectations. These changes can cascade rapidly across system architecture. Document trade-offs between privacy, performance, and explainability early to prepare for necessary compromises. Pay special attention to integration points between AI techniques, as modifications to data formats, syntax, or semantics in one module typically require adjustments in connected components. Establishing flexible adaptation protocols minimizes disruption when requirements shift.

E 1.1 Key Activities:

- Conduct stakeholder interviews to identify core requirements and data privacy constraints
- Analyze existing systems and processes that will integrate with the new AI solution
- Document domain-specific workflows, identify possible AI integration points
- Identify regulatory compliance requirements applicable to the domain
- Classify data sensitivity levels and establish initial handling protocols
- Create user personas and scenarios to understand basic interaction needs
- Catalog domain expertise and rules that might inform later AI approaches
- Document initial performance expectations and stakeholder success criteria
- Define primary objectives (critical, must-have requirements) and secondary objectives (supplementary, nice-to-have features), and prioritize them
- Document stakeholder expectations regarding AI capabilities and limitations

E 1.2 Data Analysis

E 1.2 Principles:

- **Examine Data Before Designing**

Analyzing available data reveals requirements and insights not initially considered by stakeholders, such as insufficient data volume or structures incompatible with language model fine-tuning.

- **Keep Options Open**

Consider multiple approaches for data preparation, information extraction, and AI processing. Maintaining flexibility prevents premature commitment to suboptimal solutions.

- **Identify Hybrid Integration Points Early**

Search for opportunities where symbolic and sub-symbolic approaches together overcome individual limitations. Focus on verification points, knowledge enhancement possibilities, and format enforcement needs.

E 1.2 Guidelines:

Begin by profiling data characteristics including volume, structure, quality, and sensitivity levels. This profile guides AI technique selection: structured data with explicit rules suits symbolic approaches, while large unstructured datasets benefit from sub-symbolic methods, as detailed in Section 2.

For data-sensitive domains, evaluate privacy implications of different AI techniques. Remote sub-symbolic processing, whether through GPU clusters or cloud services like *Fly.io*, may require data anonymization or synthetic data generation.

E 1.2 Key Activities:

- Perform statistical analysis of data volume, quality, distribution, and structure
- Identify data patterns, outliers, and anomalies that may influence AI technique selection
- Document data sensitivity levels and associated processing constraints
- Identify data extraction and transformation requirements
- Determine appropriate AI processing techniques for certain forms of data
- Evaluate whether data volume and quality are sufficient for fine-tuning, or if symbolic rule-based techniques or RAG approaches might be more suited
- Map domain-specific rules and constraints that could be encoded in symbolic components
- Determine scenarios where AI can produce information as part of the primary requirements
- Determine which parts of the system require high information integrity and verification
- Identify verification points where symbolic AI / human review should validate sub-symbolic AI outputs

E 1.3 Infrastructure & Resource Planning

E 1.3 Principles:

- **Start Small, Scale When Necessary**

Begin with smaller, locally deployable models before committing to resource-intensive options. Proving concepts with 8B parameter models before scaling to 70B models prevents wasted resources while maintaining upgrade paths.

- **Plan for Multiple Deployment Scenarios**

Develop fallback options for each processing tier. Data-sensitive domains require contingency plans when primary approaches prove insufficient or deployment environments change unexpectedly.

E 1.3 Guidelines:

Begin infrastructure planning by inventorying available hardware, particularly GPU memory, CPU specifications, and network connectivity. Match these resources with appropriate AI models using quantization strategies that balance performance and constraints.

Training language models requires significantly more computational resources than inference. While quantization efficiently reduces inference costs, avoid it during fine-tuning as it compromises training stability and result quality. For deployment, select the largest practical model for your hardware, typically using 4 to 8 bit quantization to maintain output quality and reasoning capabilities.

For data-sensitive domains, design a tiered processing strategy where each tier represents an option. Tier 1 uses smaller local models for highly-sensitive data, Tier 2 enables anonymized processing with larger remote models when local processing is insufficient, and Tier 3 provides cloud-based services for non-sensitive operations. Document these alternatives to ensure operational continuity.

Test target operating systems and runtime environments early, as they significantly impact technology choices. Verify compatibility of key frameworks such as Ollama¹, LangChain², and llama.cpp³ across potential deployment platforms before committing to specific technologies.

¹<https://ollama.com/>

²<https://www.langchain.com/>

³<https://github.com/ggml-org/llama.cpp>

E 1.3 Key Activities:

- Document available hardware resources, focusing on CPU, RAM, GPU specifications, storage capacity, and bandwidth and latency of internet connection
- Evaluate local deployment options for different model sizes with various quantization levels
- Assess network connectivity constraints that might impact remote AI service usage
- Identify target operating systems and compatibility requirements for deployment environment
- Document libraries, APIs, and frameworks compatible with target environment
- Develop data sanitization protocols for scenarios requiring remote processing of sensitive data
- Map stakeholder data sensitivity requirements to appropriate processing environments (local vs. remote)
- Establish fallback processing strategies when primary approaches prove insufficient
- Determine specialized hardware needs for specific AI techniques (e.g., requirements for fine-tuning)
- Estimate inference latency requirements and match with appropriate model configurations

E 1.4 System Outline

E 1.4 Principles:

- **Build Modular, Not Monolithic**
Create self-contained modules for each core functionality rather than one interconnected system. This enables independent development, testing, and maintenance while allowing modules to evolve at different rates.
- **Keep Module Boundaries Flexible**
Design adaptive boundaries between components that accommodate changing requirements throughout development. This allows the hybrid AI system to evolve without major architectural redesigns.
- **Coordinate Through the Orchestration Layer**
Create a consistent orchestration layer with standardized data pipelines for module communication. Modules can use specialized internal implementations while maintaining system coherence through unified data flows.

E 1.4 Guidelines:

Design modules based on functional domains or data processing flows rather than technical implementation. This ensures modules align with business requirements and can evolve as technologies change.

Create standardized interfaces and unified data pipelines for all modules regardless of internal implementation. Each module should expose consistent APIs for inputs and outputs while internally using the most appropriate technologies, whether symbolic AI, LLMs, or rule engines. Define common data formats, transformation patterns, and exchange protocols to ensure reliable communication between diverse AI components.

For data-sensitive domains, implement strict data compartmentalization with explicit handling boundaries between modules. Document where sensitive data enters, processes, and exits the system. Apply the principle of least privilege and need-to-know access, minimizing modules with access to sensitive information. When using remote functionalities like cloud processing, add safeguards including data anonymization and encryption for third-party connections. Plan for both vertical scaling, improving individual module capabilities, and horizontal scaling, adding new modules. Document extension points for future modules to enable system growth without architectural overhaul.

E 1.4 Key Activities:

- Map stakeholder requirements to specific system modules
- Create a high-level system diagram depicting core modules and their interactions
- Identify key decision points where symbolic and sub-symbolic approaches will be integrated
- Define the orchestration strategy for coordinating module activities
- Define clear interfaces between modules, focusing on data exchange formats and protocols
- Document data flow pathways between modules, highlighting where sensitive information is processed
- Design standardized and reusable data pipelines for cross-module communication and information exchange
- Develop a data transformation strategy for cross-module communication
- Create technical specifications for each module's responsibilities and constraints
- Document technology stack decisions for each module and the orchestration layer
- Establish error handling and fallback procedures between modules
- Identify potential bottlenecks or performance concerns at the architectural level

Phase 2: Design & Implementation Preparation

Phase 2 begins the iterative cycle through Phases 2 through 4 or 5. Each iteration can focus on a single module or modify multiple modules simultaneously when system refinements affect shared data formats, syntax, or semantics. This phase provides guidance for data preparation, whether for language model fine-tuning or symbolic AI construction.

E 2.1 Module Selection & Planning

E 2.1 Principles:

- **Build Core Features First**

Select modules delivering core requirements before peripheral features. This approach front-loads value delivery and establishes critical functionality early while creating a foundation for iterative enhancement.

- **Define Clear Module Boundaries**

Specify module responsibilities according to the system outline with explicit input/output definitions. Maintain strict functional boundaries to prevent scope creep during planning.

- **Prototype Before Committing**

Create experimental implementations for uncertain components before finalizing architecture. Prototypes validate technical approaches and reveal integration challenges early.

E 2.1 Guidelines:

Use dependency graphs to identify logical development sequences. Prioritize modules providing core services or fundamental data transformations before dependent modules.

Expand the system outline into detailed technical specifications defining how modules accomplish their tasks. For high-risk or uncertain components, develop low-fidelity prototypes to validate assumptions. These prototypes both confirm technical feasibility and inform contingency planning. Document alternative approaches for risky components to ensure development continues if initial strategies fail.

Test APIs, libraries, and third-party services with representative data before integration. Verify parameter handling, response formatting, and module compatibility to prevent integration surprises and confirm external dependencies function as promised.

E 2.1 Key Activities:

- Create a module dependency graph to establish logical implementation sequence
- Prioritize modules based on combined primary objectives and technical risk assessment

- Define detailed module responsibilities, expanding on the system outline from Phase 1
- Identify specific integration points with existing system components
- Document specific symbolic and sub-symbolic components within each selected module
- Document integration points where control transfers between symbolic and sub-symbolic components
- Define specific data formats and transformation rules for intra-module communication
- Establish concrete success criteria and evaluation metrics for this iteration
- Document specific third-party libraries, APIs, and frameworks to be used in implementation
- Develop low-fidelity prototypes specifically targeting high-risk technical approaches
- Devise contingency plans for modules with high implementation uncertainty
- Create a detailed development roadmap with milestones for the current iteration

E 2.2 Data Preparation & Enrichment

E 2.2 Principles:

- **Prioritize Data Quality Over Quantity**

Focus on data accuracy, consistency, and relevance rather than volume. Clean, well-structured data yields better results for both symbolic rule extraction and sub-symbolic fine-tuning than larger amounts of noisy information.

- **Build In Privacy From the Start**

Integrate privacy protection into data workflows from the beginning. For data-sensitive domains, this proactive approach ensures compliance while maintaining data utility for AI development.

E 2.2 Guidelines:

Begin data preparation by implementing the sanitization procedures for PII and sensitive content identified in Phase 1. When using remote technologies like cloud GPUs or third-party services, apply planned anonymization or synthetic data generation to minimize sensitive data exposure.

Transform stakeholder data into machine-processable formats, including plain text (TXT) files or structured Markdown (MD) files. Apply systematic normalization to

eliminate noise and redundancies while preserving contextual meaning. Address conversion artifacts such as character encoding issues, structural corruption, or metadata loss. Based on Phase 1 model specifications, establish context window constraints for fine-tuning datasets. Limit individual files to 80 % of the model's maximum token capacity, allocating 10 % for task templates and instructions, with a 10 % safety margin for tokenizer variations. This conservative approach ensures processing stability and prevents truncation during training and inference.

Extract high-value information through targeted techniques that leverage domain expertise. Generate derivative datasets highlighting domain-specific patterns, edge cases, and specialized contexts relevant to module development. This extracted information enhances module functionality, enriches fine-tuning data, and creates comprehensive evaluation datasets. For sensitive domains, ensure extraction processes maintain Phase 1 privacy constraints.

Document all transformation decisions, filtering criteria, and extraction rules for reproducibility and transparency. For complex datasets, create hierarchical pipelines progressing from general cleaning through domain-specific transformations to module-specific optimizations. Develop validation procedures confirming semantic equivalence between original and transformed data to prevent information loss or distortion.

E 2.2 Key Activities:

- Implement the PII detection and sanitization procedures defined during infrastructure planning
- Configure context window partitioning for fine-tuning datasets (80 % content)
- Develop document parsing workflows specific to the selected module's input requirements
- Create data normalization procedures that standardize formats while preserving semantic meaning
- Develop data filtering mechanisms to remove noise and irrelevant content
- Create structured data schemas optimized for the selected module's processing needs
- Design hierarchical processing pipelines with general cleaning, domain-specific transformations, and module-specific optimizations
- Document specific transformation rules and preprocessing decisions for reproducibility
- Create data validation procedures that verify extraction and transformation accuracy
- Implement entity and relationship extraction procedures targeting domain-specific concepts
- Generate augmented datasets with contextual metadata to enhance training signal

E 2.3 AI Component Configuration

E 2.3 Principles:

- **Choose AI Techniques Based on Module Needs**

Select AI approaches based on defined module requirements rather than technological preferences. This ensures each module uses techniques best suited for its specific tasks.

- **Confirm AI Capabilities Early**

Test AI components with representative data before full implementation to prevent significant rework later in development.

E 2.3 Guidelines:

Before implementing modules from the *Module Selection & Planning* stage, validate candidate AI approaches through focused experiments with domain-specific data. For language models, test performance across different parameter settings, including temperature and context utilization, using prompts resembling production use cases with data from the *Data Preparation & Enrichment* stage. This experimentation identifies optimal configurations and limitations while informing parameters for subsequent fine-tuning.

For hybrid AI modules, test integration between symbolic and sub-symbolic components using representative data samples. This early validation prevents compatibility issues and ensures smooth transitions between different AI approaches.

E 2.3 Key Activities:

- Test language models with task-specific prompts that align with the module's primary objectives
- Evaluate and document optimal model parameters (temperature, context length) using representative domain data
- Test API behavior with sample data to identify limitations or discrepancies from documentation
- Create prototype prompt templates for language model components aligned with module requirements
- Update integration points between symbolic and sub-symbolic components
- Implement sample symbolic rules to evaluate their effectiveness for the module's tasks
- Establish fallback mechanisms for when primary AI approaches underperform
- Document specific limitations discovered during component testing

Phase 3: Implementation & Integration

This phase transforms the outlined modules and tested AI techniques into working implementations. Modules requiring fine-tuned models begin with instruction dataset building and the fine-tuning process. These fine-tuned models then serve as foundations for remaining module components, whether standalone or combined with symbolic AI techniques.

E 3.1 Dataset Building & Fine-Tuning

E 3.1 Principles:

- **Match Training to Real Use**

Design instruction templates that mirror the prototype prompts from AI Component Configuration. This alignment ensures models learn to respond appropriately to the exact interaction patterns they will encounter in deployment.

- **Keep General Skills While Specializing**

Construct datasets emphasizing domain-specific knowledge while preserving general task capabilities. This balance prevents catastrophic forgetting while enabling meaningful specialization.

E 3.1 Guidelines:

Fine-tuning datasets require strict consistency between training instructions and inference prompts to ensure effective knowledge transfer to production. Structure datasets according to evaluation metrics from stage *Module Selection & Planning*, allocating 70–80 % for training, 10–20 % for validation, and 5 – 15 % for testing. This distribution provides sufficient learning data while reserving samples for validation and evaluation. Format each instruction using conventions that tested well during component configuration, such as [INST] / [/INST] for Llama models or ### for Mistral models.

Incorporate enriched data from stage *Data Preparation & Enrichment*, using extracted entity relationships and domain patterns to create contextually rich instructions. Apply the same data processing pipelines developed during preparation.

Domain-specific fine-tuning should leverage successful task prompts from component testing to create instructions addressing domain knowledge retrieval, format adherence, and reasoning within domain constraints, including specific rule compliance.

For hybrid AI systems, create specialized instructions teaching models to recognize boundaries between symbolic and sub-symbolic components. Include examples where models acknowledge uncertainty or identify cases requiring symbolic verification.

E 3.1 Key Activities:

- Convert the prototype prompt templates into standardized instruction templates
- Design instruction templates that specifically address domain-specific formatting requirements
- Document rationale for instruction template design choices for reproducibility
- Incorporate the enriched data and extracted relationships from into training examples
- Include counter-examples that teach the model to recognize and acknowledge task limitations and unsolvable requests
- Create synthetic edge cases to improve model robustness on rare scenarios
- Create a balanced distribution of instruction types across domain knowledge, format compliance, and reasoning categories
- Segment dataset construction into test, validation, and training sets
- Calculate token distribution statistics to ensure compliance with the maximum token length limit
- Set optimal fine-tuning parameters based on model architecture, dataset characteristics, and hardware resource capabilities
- Establish multi-stage fine-tuning workflow if needed, based on instruction types
- Conduct fine-tuning and analyze resulting metrics

E 3.2 Implementation of Symbolic AI Components

E 3.2 Principles:

- **Combine Symbolic with Sub-symbolic Strengths**
Design symbolic components to address limitations found in sub-symbolic approaches during testing. Focus on explicit rules, formal verification, or logical reasoning where sub-symbolic methods struggle, such as strict format enforcement or complex conditional logic.
- **Make Symbolic Processing Transparent**
Implement symbolic components with explicit rule tracing and decision explanation capabilities. This transparency enables behavior verification, facilitates debugging, and provides accountability in data-sensitive domains.

E 3.2 Guidelines:

Symbolic AI implementation begins with analyzing module requirements from Phase 2 and framework elements from Phase 1 to select appropriate approaches. These components function either as standalone solutions or as complements to sub-symbolic methods.

Standalone symbolic implementations focus on capturing domain expertise through formal knowledge representation and explicit rules. In hybrid systems, performance metrics and limitations from sub-symbolic testing reveal enhancement opportunities. Common patterns include input validation and preprocessing, rule-based output generation or verification, grammar-based format enforcement, and formal reasoning for complex conditional logic.

Rule-based systems should start with minimal core rules addressing primary objectives and documented model limitations. Prioritize rules directly impacting primary objectives rather than encoding entire domain knowledge.

Knowledge representation formalism depends on specific verification tasks from component testing. Frame-based representations suit structured domain concepts, production rules handle conditional logic and business processes, while formal logic provides rigorous verification for critical decisions. Document representation choices with explicit rationale linking them to addressed sub-symbolic limitations.

For remote processing in data-sensitive domains, implement symbolic verification components that validate sub-symbolic outputs against privacy and compliance constraints without accessing original sensitive data. These components operate on abstracted or anonymized representations while providing meaningful verification. In hybrid modules, testing sub-symbolic components before implementing symbolic components identifies output patterns and edge cases early. This sequence allows symbolic components to be designed with concrete understanding of their processing requirements.

E 3.2 Key Activities:

- Identify primary objectives that symbolic components should address based on earlier phase requirements
- For hybrid implementations, analyze sub-symbolic model performance to identify specific limitations addressable by symbolic methods
- Develop a minimal set of core rules directly addressing primary objectives
- Implement rule-based validation for format consistency and structural requirements
- Create symbolic reasoners for handling complex conditional logic and multi-step inference
- Develop pattern matching components for structured information extraction and normalization

- Implement domain-specific grammar enforcement for outputs requiring precise formatting
- Implement constraint satisfaction components for scenarios with multiple interdependent requirements
- Create audit logging mechanisms that track symbolic component decisions for accountability and traceability
- Implement symbolic AI logic as generalizable as possible and inside stand-alone components

E 3.3 Unit and Integration Testing of Components

E 3.3 Principles:

- **Test Individual Components Before Integration**

Validate each AI component independently before combining them in hybrid configurations. This isolates issues to specific components and establishes baseline performance expectations.

- **Include Edge Case Testing**

Focus test cases on boundary conditions where components are most likely to fail, particularly at transition points between symbolic and sub-symbolic processing. Edge cases often reveal integration issues not apparent during unit testing.

E 3.3 Guidelines:

Sub-symbolic component testing requires consistent application of parameters identified during AI Component Configuration, including temperature, top- p , and other sampling settings. This consistency ensures performance differences reflect actual model behavior rather than parameter variations. Test cases should cover the full input range, emphasizing edge cases that might trigger undesired behaviors. Fine-tuned models require both automated evaluation using established metrics and manual evaluation of format adherence, reasoning quality, and domain alignment.

Symbolic component testing involves test suites verifying rule execution correctness, logical consistency, and boundary condition handling. Tests must confirm that symbolic components implement formal specifications from earlier phases. Document edge cases where symbolic reasoning produces unexpected results, particularly in complex rule interactions or ambiguous inputs.

Hybrid AI modules require integration tests targeting handoff points between symbolic and sub-symbolic components. These tests verify output-input compatibility, semantic consistency, and error handling across components. Create scenarios triggering fallback mechanisms to ensure proper execution when primary approaches fail. Use logging to capture internal component states, facilitating debugging of complex interactions.

Document all limitations and edge cases producing failures or unexpected results. These documented limitations guide safeguard implementation and system refinement in subsequent iterations. Data-sensitive domains require specific tests verifying privacy constraint maintenance throughout processing stages.

E 3.3 Key Activities:

- Create comprehensive test cases covering expected input variations for each component
- Apply the same model runtime configurations identified during component testing when evaluating fine-tuned models
- Use verbose intermediate output logging that captures data state between each processing step, enabling verification that individual subcomponents are functioning correctly
- Conduct manual evaluation of outputs for reasoning quality and domain alignment
- Compare component performance against the baseline metrics established in Phase 2
- Develop test suites specifically targeting the integration points between symbolic and sub-symbolic components
- Create edge case test scenarios that deliberately push components to their limits
- Verify that privacy constraints remain intact throughout component processing chains
- Document specific cases where components produce unexpected or incorrect outputs
- Document specific integration limitations to guide subsequent refinement activities

E 3.4 Interface & Pipeline Integration

E 3.4 Principles:

- **Verify Module Connections**
Test all communication channels between new modules and existing system components before deployment. Focus on data format compatibility, event handling, and error propagation across module boundaries.
- **Keep Module Boundaries Intact**
Maintain established module boundaries during integration while making incremental orchestration layer changes that minimize disruption to existing functionality. This ensures system stability while accommodating new capabilities.

E 3.4 Guidelines:

Following successful component implementation and testing, integrate the complete module into the system's orchestration layer according to interface specifications from the *System Outline* stage in Phase 1. For hybrid AI implementations, verify that symbolic and sub-symbolic handoff points operate correctly within the overall system context, particularly when triggered by external events or within larger processing pipelines.

Multi-user systems and remote resource utilization require verification that data protections remain intact during inter-module information flow. Confirm that sanitization procedures, access controls, and privacy safeguards function correctly when modules operate within the full system rather than in isolation. This verification becomes critical when sensitive data crosses user contexts or transmits to external processing environments.

Document all integration points, data exchange formats, and event handling protocols implemented during this phase. This documentation provides essential reference for future maintenance and system extensions.

E 3.4 Key Activities:

- Implement or integrate into the standardized interfaces defined during Phases 1 and 2
- Update the orchestration layer to incorporate the new module into processing pipelines, but without modifying its original structure
- Configure event handlers to trigger appropriate module functionality
- Verify error propagation and handling across module boundaries
- Test module behavior when triggered by system events rather than direct invocation
- Document the module's implementation and all integration points
- Confirm that privacy and security constraints are maintained during cross-module data transfer

Phase 4: Evaluation & Refinement

This final phase of each development iteration encompasses testing and evaluating the complete system after module additions or updates. After implementing all projected modules, the process continues to Phase 5. When development remains incomplete, the next iteration begins from Phase 2.

E 4.1 System-Level Evaluation

E 4.1 Principles:

- **Test in Real-World Conditions**

Test the system with realistic scenarios mirroring actual stakeholder workflows rather than isolated functional tests. This reveals integration issues and user experience challenges that unit testing cannot identify.

- **Verify Requirements Are Met**

Evaluate system performance against primary objectives and success criteria from Phase 1's *Domain & Requirements Analysis*. This comparison ensures implementation addresses stakeholder needs beyond technical functionality.

E 4.1 Guidelines:

System evaluation requires assessing both technical performance and stakeholder requirement alignment. Evaluation scenarios should correspond to use cases and workflows from Phase 1, incorporating the full input range including boundary cases and problematic patterns from component testing.

System-level testing focuses on cross-module interactions rather than isolated functionality. Evaluation examines data and control flow across module boundaries, emphasizing transformation accuracy, error propagation, and recovery mechanisms. Hybrid AI systems require special attention at symbolic and sub-symbolic handoff points, where emergent behaviors often arise.

Both automated and manual evaluations should be performed. Automated testing provides consistent quantitative metrics, while manual evaluation captures qualitative aspects difficult to measure programmatically, particularly for sub-symbolic AI-generated outputs. Data-sensitive domains require verification that privacy, security measures, and user access controls function correctly at system level, with specific tests identifying potential information leakage across module boundaries.

Documentation of system-level issues requires clear categorization: critical flaws needing immediate attention, performance limitations requiring architectural changes, and minor issues addressable through parameter tuning or small adjustments. This prioritized documentation guides subsequent refinement planning.

E 4.1 Key Activities:

- Create evaluation scenarios directly mapped to original stakeholder requirements
- Conduct end-to-end testing across complete processing pipelines
- Measure system performance metrics including response time, throughput, and resource utilization
- Analyze processing bottlenecks across module boundaries
- Evaluate data transformation accuracy across the full processing chain
- Perform specific tests targeting system behavior under high load or stress conditions
- Conduct information integrity verification tests for hybrid AI scenarios targeting the minimization of hallucinations or missing information
- Compare actual performance against the success criteria established in Phase 1
- Document all identified limitations, categorized by severity and required effort to address
- If possible, collect feedback from stakeholders to validate alignment with expectations
- Analyze gaps between expected and actual performance to inform refinement planning

E 4.2 Refinement Planning

E 4.2 Principles:

- **Address High-Impact Issues First**
Prioritize fixes based on objective impact on primary system objectives rather than addressing all limitations equally. Focus on issues that affect core functionalities before tackling peripheral concerns.
- **Requirements Drive Refinements**
Each planned refinement must directly address specific stakeholder requirements or system limitations from the *System-Level Evaluation*. This connection maintains focus on business value over technical perfection.

E 4.2 Guidelines:

Refinement planning begins by revisiting primary and secondary objectives from Phase 1's *Domain & Requirements Analysis*. Issues from *System-Level Evaluation* then populate a structured prioritization matrix mirroring the initial requirements hierarchy.

Critical issues impacting primary objectives or core functionality receive highest priority, followed by those affecting secondary objectives or performance, with minor enhancements deferred when resources are limited. This approach maintains alignment with original stakeholder needs.

Each high-priority issue requires a specific remediation plan detailing technical approach and estimated effort. Issues may need localized adjustments such as parameter tuning and rule modification, or significant architectural changes including component redesign and integration reconfiguration. Minor issues involving only configuration parameters or localized code receive immediate attention rather than deferral, preventing technical debt accumulation while maintaining momentum.

Iteration planning identifies the next module or objective for implementation. Remaining requirements from Phases 1 and 2 guide selection based on business value, existing component dependencies, and implementation complexity. Dependencies between refinement tasks and new module development establish logical implementation sequences, preventing wasted effort and ensuring systematic improvement.

The cumulative scope of refinements and next-module implementation determines whether to initiate a new development iteration returning to Phase 2, or proceed to Phase 5 deployment. Significant multi-component redesigns require thorough review of original Phase 1 requirements and system specifications before beginning the next iteration.

E 4.2 Key Activities:

- Create a prioritized issue matrix categorizing both system limitations and unfulfilled objectives by severity and impact on primary requirements
- Map identified issues to stakeholder requirements from Phase 1
- Implement immediate fixes for minor configuration or localized code issues
- Develop specific remediation plans for high-priority issues requiring significant changes
- Review remaining requirements and select the next module for implementation, if there are no severe issues
- Estimate effort and resources required for each planned refinement and new module
- Document dependencies between refinement tasks and new module implementation
- Identify opportunities to combine refinements with new functionality implementation
- Establish concrete success and evaluation criteria for the next iteration's objectives
- Conduct a requirements review if needed before beginning the next iteration

Phase 5: Deployment & Continuous Improvement

This final phase begins after achieving all primary objectives and system readiness for deployment. The phase transitions the system from development to production while establishing continuous improvement mechanisms. Production environment testing generates feedback revealing previously unidentified requirements and refinement opportunities. Deployment continues until sufficient new requirements and enhancements accumulate, triggering a new development iteration from Phase 2. This cyclical approach ensures ongoing system evolution responding to real-world usage patterns and emerging stakeholder needs.

E 5.1 Production Environment Setup & Deployment

E 5.1 Principles:

- **Implement Comprehensive Monitoring**

Integrate robust monitoring and logging systems tracking both technical performance metrics and AI-specific indicators such as inference latency, model confidence scores, and usage patterns.

- **Use Least Privilege Access**

Configure production systems with minimum necessary access rights to data resources and external services. This principle ensures data-sensitive domain security while maintaining functional system operation.

E 5.1 Guidelines:

Production deployment establishes the operational environment for serving end users. This requires configuring all dependencies, libraries, and runtime requirements from development within production infrastructure. Data-sensitive domains need careful attention to production-specific security configurations, including access controls, encryption settings, and sanitization procedures that often differ from development due to scale, user patterns, or regulatory requirements.

Deployment documentation maps development configurations to production settings, explaining intentional differences. Document version specifications, model quantization levels, and fixed AI component parameters. Verification procedures should test the complete system in controlled environments mirroring production conditions, using virtual machines or staging environments before actual deployment.

Monitoring systems provide ongoing detection of performance degradation and unexpected behaviors. This visibility transforms stakeholder-reported issues into specific technical limitations, generating actionable data for systematic improvements.

E 5.1 Key Activities:

- Create an environment specification document listing all dependencies and version requirements
- If required, implement access control mechanisms aligned with data sensitivity requirements
- Configure model serving infrastructure with appropriate quantization and runtime parameters
- Verify that all symbolic AI rules and constraints remain correctly implemented, and update if needed
- Document production-specific configuration settings and their rationale
- Implement comprehensive monitoring and logging for both technical metrics and AI-specific indicators
- Test system resource consumption under various load conditions to verify production stability, especially for AI components
- Conduct an initial health check verifying all components are operational
- Verify that users can access the system without any issues related to local access rights or firewall rules

E 5.2 User Testing & Feedback Collection

E 5.2 Principles:

- **Observe Actions, Not Just Opinions**
Prioritize direct observation of user interactions over self-reported experiences. This reveals actual usage patterns and challenges users may not articulate in formal feedback.
- **Focus Feedback on Core Objectives**
Structure feedback collection to assess primary objective fulfillment rather than gathering generalized impressions. This approach delivers specific directions for meaningful improvements.

E 5.2 Guidelines:

User testing employs a two-phase approach combining structured assessment with open exploration. Task-oriented scenarios mapped to primary objectives from stage *Domain & Requirements Analysis* verify that core functionalities align with stakeholder expectations under realistic conditions.

Feedback collection uses complementary methods: structured evaluation forms for quantitative performance assessment, semi-structured interviews capturing qualitative

experiences, and direct observation revealing actual usage patterns. Hybrid AI systems require particular attention to user perceptions of reliability, transparency, and output appropriateness. Instances of user surprise or confusion often reveal misalignments between mental models and system functionality.

Consistent metrics for feedback categorization enable quantitative tracking across test sessions. These metrics link directly to primary objectives and technical decisions, facilitating precise refinement planning. Unexpected usage patterns where users employ the system beyond design intentions often reveal valuable enhancement opportunities for subsequent iterations.

E 5.2 Key Activities:

- Create task-based scenarios that directly test alignment with primary objectives
- Develop structured evaluation forms that assess system performance against defined metrics
- Conduct observation sessions documenting user interaction patterns and pain points
- Implement feedback collection mechanisms directly within the production system
- Perform semi-structured interviews focusing on alignment with user workflows
- Document unexpected usage patterns and creative applications of system capabilities
- Analyze user-generated data to identify patterns and improvement opportunities
- Create systematic categorization of feedback items linked to system components
- Reassess primary objectives with stakeholders based on real-world usage insights
- Collect new requirements and refinement opportunities revealed through actual use
- Document regularly occurring errors or confusion points in user interactions
- Evaluate outputs quality on representative stakeholder tasks and expectations

E 5.3 Continuous Monitoring & Improvement

E 5.3 Principles:

- **Evolve Based on Usage Patterns**

Systematically capture user interactions and transform them into structured datasets for improving both symbolic and sub-symbolic components. This continuous feedback loop enables system evolution based on authentic usage patterns and resilience against concept drift.

- **Maintain Version Control for AI Components**

Track and manage different versions of models, rules, and configurations in production to enable safe rollbacks and A/B testing. This versioning strategy allows performance comparison over time and provides safety nets when deploying updates.

E 5.3 Guidelines:

Analysis workflows aggregate usage statistics across key dimensions including response accuracy, processing time, resource utilization, and user satisfaction. Patterns diverging from development expectations often highlight valuable refinement opportunities. Resource consumption patterns deserve particular attention for identifying optimizations that reduce operational costs while maintaining or improving performance.

Hybrid AI systems benefit from reusing data transformation pipelines to convert user data into training materials for both component types. Fine-tuned models require datasets from successful interactions exemplifying desired behavior, particularly in previously challenging scenarios. Symbolic components need analysis of manual overrides or unexpected results to refine logical rules and constraints.

Parallel monitoring of system usage and emerging technologies involves regularly evaluating new AI models, libraries, and frameworks against existing implementations. Technical horizon scanning combined with periodic stakeholder interviews identifies evolving requirements beyond usage data. Clear thresholds trigger refinement planning when sufficient improvements accumulate or transformative technologies emerge. These thresholds consider both refinement volume and potential impact on primary objectives. Meeting these conditions initiates structured transition to Phase 2, ensuring production insights inform the next development iteration.

E 5.3 Key Activities:

- For multi-user systems, configure anonymized data collection for successful user interactions and error cases
- Implement statistical analysis processes for identifying patterns across user interactions
- Implement usage pattern analysis to identify potential refinements for symbolic AI components
- Monitor resource utilization patterns to identify optimization opportunities
- Create periodic review sessions with stakeholders to evaluate collected improvement opportunities
- Document emerging user workflows that differ from originally anticipated patterns
- Establish quantitative thresholds for initiating new development iterations
- Schedule regular technology assessments to evaluate emerging AI techniques and frameworks

- Conduct periodic stakeholder interviews to identify evolving requirements and expectations
- Update the primary requirements document with insights gained from production usage, technological advancements, and stakeholder feedback
- If applicable, reuse existing data transformation pipelines for converting user interactions into fine-tuning datasets
- Conduct a structured review verifying that Phase 1 requirements and architectural decisions remain valid before initiating a new development iteration from Phase 2

References

- [Garcez *et al.*, 2008] Artur S. d’Avila Garcez, Luís C. Lamb, and Dov M. Gabbay. *Neural-Symbolic Cognitive Reasoning*. Springer Publishing Company, Incorporated, 1 edition, 2008.
- [Kambhampati, 2020] Subbarao Kambhampati. Challenges of human-aware ai systems. *AI Mag.*, 41(3):3–17, September 2020.
- [Sarker, 2021] Iqbal H. Sarker. Deep cybersecurity: A comprehensive overview from neural network and deep learning perspective. *SN Comput. Sci.*, 2(3), March 2021.
- [Sarker, 2022] Iqbal H. Sarker. Ai-based modeling: Techniques, applications and research issues towards automation, intelligent and smart systems. *SN Computer Science*, 3(2):158, February 2022.